P3482R0
Design for C++ networking based on IETF TAPS

Published Proposal, 2024-10-14

This version:
http://wg21.link/Pxxxx

Authors:
Thomas Rodgers (Woven By Toyota)
Dietmar Kühl (Bloomberg)

Audience:
SG4

Project:
ISO/IEC 14882 Programming Languages — C++, ISO/IEC JTC1/SC22/WG21

Source:
https://github.com/rodgert/papers/blob/master/source/p3185r0.bs

---

Abstract

Design proposal for creating IETF TAPS based network connections

Table of Contents

§ 1. Background

[p2762] Proposes a modification to the [netts] to adopt the Sender/Receiver model (targeted to C++26).

[p3185] Proposed a direction for C++ networking based on IETF TAPS. The process of obtaining a connection in the TAPS model is substantially different from other approaches based on Berkeley Sockets or abstractions over the socket oriented model such as the [netts] and [asio].

[p1861] sketched out an interface proposal derived from Apple's Network.Framework [ANF] and to which TAPS has design lineage. The design discussed here differs significantly in the detail, but presents many of the same concepts.

§ 2. The TAPS approach

IETF TAPS uses a property based approach for describing the requirements of a connection. The TAPS infrastructure uses these properties to select a set of one or more candidates that satisfy the supplied properties. These candidates can be then be used to request a connection which then conforms to the supplied properties. Applications may provide additional logic to select among multiple candidates where required to, e.g. provide a failover path in case of connection failure.

§ 2.1. Preconnections

Creating a connection begins with a preconnection.

TAPS specifies four groups of properties which define a preconnection object -

- Local Endpoint
- Remote Endpoint
- Transport
- Security

§ 2.2. Endpoints

Endpoints define the origin and destination points for a connection and are distinguished into local and remote types. Endpoints have the following properties -

- hostname, ex. "nyarlathotep.example.org"
- interface, ex. "en0"
- service, ex. "https"
- multicast_group, ex. "224.0.0.252" or "ff02::114"
- hop_limit, ex. 2

§ 2.3. Transports

Transports are defined by a set of requirements that the underlying infrastructure is expected to satisfy. Most transport requirements are expressed as preferences which can take one of the following values -

- Require
- Prefer
- None
- Avoid
- Prohibit

Transports have the following properties which can be used to express requirements -

- interface, a multivalued set of (interface, preference) tuples
- reliability, preference
- preserve_msg_boundaries, preference
- per_msg_reliability, preference
- preserve_order, preference

- zero_rtt_msg, preference
- multistreaming, preference
- full_checksum_send, preference
- full_checksum_recv, preference
- congestion_control, preference
- keep_alive, preference
- use_temp_local_address, preference
- multipath, enumeration { disabled, active, passive }
- advertises_alt_addr, preference
- direction, enumeration { both, send, recv }
- soft_error_notify, preference
- active_read_before_send, preference

§ 2.4. Security

Security for transports are also defined by a set of requirements that must be satisfied. The following properties are supported -

- allowed_security_protocols, sequence of of protocol identifiers
- server_cert, sequence of certificates
- client_cert, sequence of certificates
- pinned_server_cert, sequence of certificates
- alpn, sequence of application layer protocol negotiation values
- supported_group, sequence of group identifiers
- ciphersuite, sequence of ciphersuite identifiers
- signature_algorithm, sequence of algorithm identifiers
- max_cached_session, integer
- cached_session_lifetime, duration
- preshared_key, key material
- trust_verification_handler, sequence sender
- challenge_handler, sequence sender

§ 2.5. Preconnections and Connections

The endpoint, transport, and security property sets are used to create a preconnection. Preconnections are then used to establish connections. TAPS defines three methods on a preconnection type for creating a connection -

- Initiate -> Connection, an *Active Open* connection (could be named `client_connection`)
- Listen -> Connection, a *Passive Open* connection (could be named `server_connection`), the returned connection type permits rate-limiting of inbound connections by setting a connection limit property which is decremented on each new inbound connection, and may be periodically reset.
- Rendezvous -> Connection, a *Peer to Peer* connection (could be named `peer_to_peer_connection`)

TAPS further specifies that a preconnection can be modified, but such modifications only have effects on subsequent connections.

TAPS specifies that preconnections can perform endpoint resolution -

- Resolve -> (local_endpoint[], remote_endpoint[])

§ 2.6. Message Based

TAPS is explicitly message based. This is a departure from Berkeley Sockets or the [netts] which yield up buffers as data arrives, leaving framing up to the application. TAPS introduces the notion of a *Framer* which produces complete messages (or an error) from one or more chunks of received data. Framers are an optional argument to obtaining a Connection from a Preconnection. Framers are event driven, receiving events about connection initiation, incoming and outgoing data, and connection termination, from the underlying transport.

Framers allow extending a connection's protocol stack to define how to encode outbound messages, decode inbound messages, and provide well-defined message boundaries even when using stream-oriented transport protocols.

A `default_framer` could behave similarly to typical Berkeley socket code and yield the octets received thus far. Higher level examples of framers might include an `http_framer` that would parse HTTP header properties and message boundaries from the underlying stream of bytes. User defined framers could do things like using Thrift serialization/deserialization to operate on strongly typed domain types rather than spans of octets.

§ 2.6.1. Message Contexts

On calls to send and receive data, the application may provide a MessageContext. On completion of a Send or Receive operation, the event handler is provided the MessageContext associated with that event. These MessageContexts serve several functions. A MessageContext passed to Send or Receive may communicate framer-specific properties that control how a framer operates on the message data. The MessageContext can be used to communicate with the eventual `set_value()` handler receiving a completed message. MessageContexts are also used to correlate multiple partial Send or Receive operations. On receipt of octets from the transport, the MessageContext will contain information about the connection such as interface, remote endpoint, and so forth. Framers may extend the properties of the message context to include additional information, e.g., HTTP headers, which are metadata about the message being framed.

§ 3. Design discussion

The proposed general direction is to prefer TAPS concepts where sensible, e.g., in describing the properties of a connection, being message oriented, user extensible framing and message types, etc. but to otherwise adopt and amend as much of [p2762] as is applicable to form the basis of the overall proposal.

For instance, [p2762] defines a socket type(s) and a range of CPOs defining the operations over that type. Rather than a socket, the `connect()`, `listen()` and `rendezvous()` CPOs proposed here return a `connection` for which CPOs like `async_send()` and `async_receive()` would operate on much as [p2762] envisions them working on a socket type. Certain socket-oriented operations in [p2762] would not make sense though, for instance, anything related to building a socket acceptor is part of the connection type returned by the `listen()` CPO and is library implementation detail that is not exposed to the library user.

The committee typically defers to LEWG as the ultimate bikeshedder/arbiter of naming things, however SG4 should provide guidance and encouragement to LEWG in choosing names and concepts to avoid deviating unnecessarily from [TAPS_Arch] so that there is a common vocabulary with other TAPS implementations.

Most of the properties specified by TAPS assume that the underlying transport is based on TCP/IP. The discussion at the Tokyo meeting included speculation on how other transports might be supported by a proposal based on TAPS, for instance using MPI as an underlying transport, which may in turn use some form of high performance fabric, e.g., Infiniband. Similarly, [asio] provides abstractions for sockets which are based on serial ports, IPC queues, and so forth. An endpoint for an IPC queue would not include a hostname, for instance. Security properties would also not be relevant to the typical IPC

`connection` as this is enforced locally by the host operating system. Transport properties might be similarly optional, or implied by the endpoint/transport type.

[TAPS_interface] envisions three distinct kinds of connection arising from the properties of a preconnection -

- Active Open - models a typical client connection to a server
- Listen - models a typical server connection (socket acceptor pattern)
- Rendezvous - models a typical peer-to-peer arrangement

If we allow for support of non-TCP based transports, not all of these types of connection may be supported by an underlying transport. A publish/subscribe transport (e.g., Tibco Rendezvous), for instance, might only support peer-to-peer connections.

TAPS expects that a connection is a single object type that exposes all of the necessary operations to cover each of the distinct types of connection. This paper instead proposes that each kind of connection be a distinct type, exposing only those operations supported by the underlying transport for that type of connection. It is further proposed that the various connect operations be customization points, based on the type of the supplied preconnection argument, permitting vendor/user customization for supporting non-TCP/IP based transports.

The Standard Library would provide a default transport implementation that conforms to the endpoint, transport, and security properties outlined by TAPS, segregating those types specific to TCP/IP into a `net::ip` namespace similar to how the [netts] and [asio] organize TCP/IP specific types.

Preconnections are constructed from sets of properties that describe the endpoints, transport, and security requirements. [p3325] describes a mechanism for constructing and querying strongly typed properties for `std::execution` environments. Making a preconnection a template requiring its argument types to conform to the proposed queryable concept allows user code to provide any type(s) which satisfy this constraint. A related proposal being explored is a type-erased `runtime_env`, which conforms to the queryable concept. This mechanism is used here to provide the default implementations of the various endpoint, transport and security property sets.

There are corresponding property type definitions for the various properties which define a preconnection. Each property type and the sets of the properties; `local_endpoint`, `remote_endpoint`, `transport_props`, and `security_props` have value semantics. The sets of properties; `local_endpoint`, `remote_endpoint`, `transport_props`, and `security_props` conform to the `has_property` or `maybe_has_property` concept.

For transport and security properties, the standard library should provide common and well formed defaults that the user may opt into entirely, or combine with specific property choices for their use case.

These could be exposed as static members of the `transport` and `security` property sets.

The set of properties allowed should be open-ended. This paper doesn't contemplate how an implementation might make use of this flexibility, other than to propose that `connect()`, `listen()`, and `rendezvous()` be expressed as CPOs, which would allow customization over how the properties of a preconnection are processed.

The security handler properties for trust verification and challenge handling are also inherently asynchronous operations. [TAPS_interface] envisions these would be typical callback closures, but these operations can and should be represented using sender/receiver vocabulary types and participate in timeout and cancellation.

A Future paper revision will more fully cover a discussion of sequence senders. The concept of a `sequence_sender` extends the `sender` by adding a `set_next_value()` operation, which itself returns a sender which may be scheduled for later execution when it receives a value by the `io_context`.

Trust verification callbacks as sequence senders - [TAPS_interface] section 6.3.8 states -

> Security decisions, especially pertaining to trust, are not static. Once configured, parameters can also be supplied during Connection establishment. These are best handled as client-provided callbacks. Callbacks block the progress of the Connection establishment, which distinguishes them from other events in the transport system. How callbacks and events are implemented is specific to each implementation. As noted, these callbacks are intended to block the forward progress of establishing a connection. Indeed, these callbacks may require input from a user dialog in a UI, and remain blocked indefinitely if the user strolls off for a good strong cuppa, while somewhere in the bowels of the networking implementation an `io_executor` is waiting for a response to proceed with the connection. Basing these operations on `sequence senders` allows this to be efficiently scheduled and potentially cancelled.

The two security callback types are -

- Trust verification, e.g., validating a remote endpoint's X.509 certificate
- Identity challenge, invoked whenever a private key operation is required

These callbacks, when modeled as a `sequence_sender` receive a `set_next_value()` delivering the value required to perform the trust verification or identity challenge, returning a sender on which the user code will later call `set_value()`, `set_error()`, or `set_stopped()` indicating the outcome of the operation. This design also permits multiple/multi-step invocations of these callback operations, rescheduling the connection state machine with the `io_context` as needed.

Framers, in addition to producing a *body type* for a message type, may introduce headers. Headers can be strongly typed, and the `runtime_env` mechanism being explored for the preconnection property sets can also be used for this purpose. A standard set of headers derived from the properties used to construct a connection are provided to the framer. Framers can be viewed as a stack, refining one or more partial chunks of received data into a concrete message type. Our proposal is to supply a `default_framer` which provides raw octets of the message body as they are received, along with the default set of message properties derived from the connection. User defined framer types may consume the messages provided by the `default_framer` and produce strongly-typed messages.

Connection negotiation and establishment, framing, message sending, and receipt are all inherently asynchronous operations in TAPS. The C++ design for these features should use the sender/receiver vocabulary targeted at C++26, but that proposal is insufficient to cover the networking use case and there needs to be additional work in refining the proposed `sequence_sender` concept.

Connections expose a set of connection specific properties. Connection properties are modifiable, and may be modified both during pre-establishment and after the connection has been established.

[TAPS_interface] suggests that `resolve` should be a synchronous operation on the preconnection, returning a pair of collections of local and remote endpoints. DNS lookups are UDP based and the classic approach to DNS resolution is an asynchronous operation, which may return multiple results across multiple UDP DNS responses. The widely used [c-ares] DNS resolver library expresses DNS resolution as an asynchronous operation as well. It is also possible that name resolution can block indefinitely. A typical approach to address this possibility might be to provide a timeout to the `resolve()` call , e.g., `try_resolve_for()`, but there are other scenarios where one might want to abandon a query early. DNS queries can potentially return multiple responses, for instance. An application might not care to wait for a complete set of responses before initiating a connection, and upon successful establishment, abandon waiting for any further query results. This suggests that we should instead express resolution in terms of returning a Sender, rather than synchronously waiting for resolution and making that process cancellable via `stop_token`.

The process of constructing a Connection typically relies on resolving endpoints by name. If `resolve` is asynchronous, returning a sender which ultimately delivers endpoints or an error, the entry points that create connections similarly return senders which ultimately deliver a connection or an error.

Connections are a sender of asynchronous events about the connection life-cycle in addition to events for message receipt and delivery. These life-cycle events are -

- 'soft errors' if the underlying protocol stack supports access to ICMP error messages related to the Connection.

- 'path change' events to notify the application of the underlying connection path, or if the set of local endpoints changes, etc.
- Closed, Abort, CloseGroup, AbortGroup, ConnectionError, etc.

Connections are a sender of framed and, optionally, strongly typed, received Messages.

[p2762] modifies the socket based approach of the [netts] to adopt the Sender/Receiver model proposed by [p2300], to support asynchronous network operations. Some of these operations are subsumed by the design approach taken by TAPS, but many others continue to be relevant, if somewhat modified, under this proposal.

In particular -

- `async_accept()` is subsumed by the proposed `async_listen()` sender operation
- `async_connect()` is subsumed by the proposed `async_initiate()` operation
- `async_resolve_address()` and `async_resolve_name()` are subsumed by the proposed `async_resolve()` operation

The `async_listen()` operation returns a sender, which may be `.connect()`-ed to receive the result of an accepted connection. The application may throttle the total rate of underlying socket accepts, by throttling the rate at which subsequent `async_listen()` operations are initiated.

The operations defined for sending data in [p2762] are modified as follows -

- TAPS is explicitly message base, `async_send()` is a customization point intended to send whole messages, files, etc. The decision as to how a message is encoded to an underlying transport under TAPS is delegated to a customizable Framer, which is supplied at connection establishment. TAPS message operations always include a caller supplied message context. The `async_send()` method, as defined in [p2762] is modified to accept a message context. Message framers may introduce a strongly typed message requirement, in which case the message argument to `async_send()` is that type, otherwise it is `ConstantBufferSequence`.
- The message_flags parameter to `async_send()` is removed; any properties that direct the transport's delivery of the message are supplied as part of the MessageContext.
- The return type of `async_send()` is a *send-sender*. This sender may deliver values indicating that a message has been sent, or possibly that a message send timed out before the message could be sent, in addition to any error that might arise in the process of sending the message.
- There is no difference between sending a message to a stream transport vs. a datagram oriented transport from the user's perspective so `async_send_to()` is removed.

- TAPS allows for partial send operations. Such partial sends send portions of a single logical message, which is correlated as a single logical transaction by passing the same MessageContext instance.
- Partial sends still return a *send-sender* which receives the value or error that results from sending that discrete message fragment. It is possible, under this scenario, for a partial message logical send transaction to have partial failures and timeouts. [TAPS_interface] Identifies this case and suggests that the implementation provide some way to correlate partial completions to which distinct partial send operation(s) failed. This could be accomplished by requiring the user to pass a distinct fragment ID as part of the MessageContext. This would imply that the MessageContext is copied for each partial send. Doing so may be undesirable, and other approaches might be worth exploring.

Similarly the operations defined for receiving data are modified as follows -

- TAPS is explicitly message based, `async_receive()` is a customization point intended to receive whole messages, files, and so forth. The decision as to what constitutes a message under TAPS is delegated to a customizable framer, which is supplied at connection establishment. TAPS message operations always include a caller supplied message context. The `async_receive()` method, as defined in [p2762] is modified to accept a message context.
- There is no difference between receiving a message from a stream transport vs. a datagram oriented transport from the user's perspective so `async_receive_from()` is removed.
- An open design question is whether the supplied message context could carry, for example, a buffer pool that the framer could then use to obtain buffers for the data of messages currently in the process being framed.
- TAPS allows for partial receive operations, which are similar to `async_read_some()`. In the TAPS scheme, the application issues multiple partial receive operations, indicating how many octets, at most, to receive, before delivering a partial result. All such partial receive operations specifying the same MessageContext instance are part of the same logical receive operation. The proposal here is to either extend the signature for `async_receive()` to include a maximum read length value, or introduce an `async_receive_some()` operation.
- Each call to `async_receive()` returns a *receive-sender*, which can be `.connect()`ed to receive the result of that operation. Applications can throttle connections and potentially provide back pressure by controlling the number of concurrent `async_receive()` operations in progress at any instant.
- The `set_value()` call ultimately delivered by an `async_receive()` includes the message context supplied when the operation was initiated, and the (potentially strongly-typed) message or partial message result.

- `async_receive()` operations are cancellable by the usual `stop_token` mechanism as with any other asynchronous operation under the `std::execution` proposal.

A future revision of this paper will discuss framers and the details of the underlying transport exposed to them.

§ 4. Proposed API

These types are from (generally) [p3325] and live in the `std::execution` namespace.

§ 4.1. `std::execution::property_key_list`

```
namespace std::execution {
    ...

    template<typename... Ts>
    class property_key_list { };

    ...
}
```

§ 4.2. `std::execution::is_property_key_list_v`

```
namespace std::execution {
    ...

    template<typename T>
    inline constexpr bool is_property_key_list_v = false;

    template<typename... Ts>
    inline constexpr bool is_property_key_list_v<property_key_list<Ts...>> = tr

    ...
}
```

§ 4.3. `std::execution::queryable`

This concept determines whether or not a type is a Queryable Environment.

```
namespace std::execution {
    ...

    template<typename T>
    concept queryable =
        requires
        {
            typename T::property_keys;
            requires is_property_key_list_v<typename T::property_keys>;
        };

    ...
}
```

§ 4.4. `std::execution::has_query`

This concept determines if an Environment supports a Query.

```
namespace std::execution {
    ...

    template<typename E, typename Q>
    concept has_query =
        requires (E const& env)
        { env.query(Q{ }); };

    ...
}
```

§ 4.5. `std::execution::has_query_default`

This concept determines whether or not an Environment has a default value for a given Query.

```
namespace std::execution {
    ...

    template<typename Q>
    concept has_query_default =
        requires
        { Q::default_value(); };

    ...
}
```

§ 4.6. `std::execution::property`

This concept determines what it means for a type to be a property.

```
namespace std::execution {
    ...

    template<typename T>
    concept property =
        std::is_empty_v<T>
        && std::default_initializable<T>

    ...
}
```

§ 4.7. `std::execution::has_property`

This concept determines if a Queryable has a given property.

```
namespace std::execution {
    ...
```

```
    template<typename Q, typename P>
    concept has_property =
        queryable<Q>
        && property<P>
        && has_query<Q, P>;

    ...
}
```

The types from [p3325] are extended to support a type-erased runtime Environment with possibly empty properties.

§ 4.8. `std::execution::has_try_query`

This concept determines if an Environment supports `try_query`.

```
namespace std::execution {
    ...

    template<typename E, typename Q>
    concept has_try_query =
        requires (E const& env);
        { env.try_query(Q{ }); };

    ...
}
```

§ 4.9. `std::execution::maybe_has_property`

This concept determines if a Queryable may optionally have a given property.

```
namespace std::execution {
    ...

    template<typename Q, typename P>
    concept maybe_has_property =
        queryable<Q>
        && property<P>
        && has_try_query<Q, P>;
```

```
    ...
}
```

§ 4.10. `std::execution::runtime_property`

This concept determines if a type is a runtime type-erasable property.

```
namespace std::execution {

    ...

    template<typename T>
    concept runtime_property =
        property<T>
        && requires { typename T::type_erased_type; };

    ...
}
```

§ 4.11. `std::execution::runtime_env`

A runtime type-erased Queryable Environment.

```
namespace std::execution {

    ...

    class runtime_env
    {
    public:
        runtime_env() noexcept = default;
        operator runtime_env_ref() const noexcept;

        template<runtime_property P, typename Tp>
            requires std::constructible_from<typename P::type_erased_type, Tp>
        void set(P prop, Tp&& init);

        template<runtime_property P>
```

```
        void unset(P) noexcept;

        template<runtime_property P>
        std::optional<typename P::type_erased_type> try_query(P prop) const
            noexcept(std::is_nothrow_copy_constructible_v<typename P::type_eras
    };

    ...
}
```

§ 4.12. Properties of endpoints

§ 4.12.1. `std::net::ip::address_v4`

The IPV4 Address type.

```
namespace std::net::ip {

    ...

    class address_v4
    {
    public:
        using uint_type = uint_least32_t;
        using bytes_type = std::array<unsigned char, sizeof(uint_type)>;

        constexpr address_v4() noexcept;
        constexpr explicit address_v4(bytes_type const& bytes) noexcept;
        constexpr explicit address_v4(uint_type v);
    };
    ...
}
```

§ 4.12.2. `std::net::ip::address_v6`

The IPV6 Address type.

```
namespace std::net::ip {

    ...

    using scope_id_type = uint_least32_t;

    class address_v6
    {
    public:
        static constexpr std::size_t const bytes_len = sizeof(::in6_addr::s6_ad
        using bytes_type = std::array<unsigned char, bytes_len>;

        constexpr address_v6() noexcept;

        constexpr explicit address_v6(bytes_type const& bytes,
                                      scope_id_type scope = 0)
                noexcept(std::numeric_limits<bytes_type::value_type>::max() ==
    };

    ...
}
```

§ 4.12.3. `std::net::ip::address`

The IP Address type.

```
namespace std::net::ip {

    ...

    class address
    {
    public:
        constexpr explicit address(address_v4 addr);
        constexpr explicit address(address_v6 addr);

        constexpr bool is_v4() const noexcept;
        constexpr bool is_v6() const noexcept;
```

```
    };

    ...
}
```

§ 4.12.4. `std::net::hostname`

The hostname type.

```
namespace std::net {

    ...

    class hostname
    {
    public:
        hostname(std::string_view str);

        ...
    };

    ...
}
```

§ 4.12.5. `std::net::interface`

The network interface type.

```
namespace std::net {

    ...

    class interface
    {
    public:
        interface(std::string_view str);

        ...
    };
```

```
    ...
}
```

§ 4.12.6. `std::net::service`

The network service type.

```
namespace std::net {

    ...

    class service
    {
    public:
        service(std::string_view str);

        // TODO constants for well known services, e.g. 'https'

        ...
    };

    ...
}
```

§ 4.12.7. `std::net::single_source_multicast_group_address`

The single source multicast group address type.

```
namespace std::net {

    ...

    class single_source_multicast_group_address
    {
    public:
        single_source_multicast_group_address(ip::address group, ip::address so

        ip::address const& group() const noexcept;
```

```
        ip::address const& source() const noexcept;

        ...
    };

    ...
}
```

§ 4.12.8. `std::net::endpoint_props`

Nested namespace containing strongly typed properties of endpoints.

```
namespace std::net::transport_props {

    ...

    class hostname
    {
    public:
        using value_type = hostname;
        ...
    };

    class ip_address
    {
    public:
        using value_type = ip::address;
        ...
    };

    class port
    {
    public:
        using value_type = uint16_t;
        ...
    };

    class interface
    {
    public:
        using value_type = interface;
        ...
    };
```

```
    class service
    {
    public:
        using value_type = service;
        ...
    };

    class multicast_group
    {
    public:
        using value_type = ip::address;
        ...
    };

    class hop_limit
    {
    public:
        using value_type = uint16_t;
        ...
    };

    class any_source_multicast_group
    {
    public:
        using value_type = ip::address;
        ...
    };

    class single_source_multicast_group
    {
    public:
        using value_type = single_source_multicast_group_address;
        ...
    };

    ...
}
```

§ 4.13. Properties of transports

§ 4.13.1. `std::net::transport_preference`

Enumeration to describe the preference for a transport property to apply to a connection.

```
namespace std::net {

    ...

    enum class transport_preference
    {
        require,
        prefer,
        none,
        avoid,
        prohibit
    };

    ...
}
```

§ 4.13.2. `std::net::multipath_preference`

Enumeration describing the preference for the transport to support a multipathing.

```
namespace std::net {

    ...

    enum class multipath_preference
    {
        disabled,
        active,
        passive
    };

    ...
}
```

§ 4.13.3. `std::net::direction_preference`

Enumeration describing the directionality preference for a connection.

```
namespace std::net {

    ...

    enum class direction_preference
    {
        bidirectional,
        send,
        recv
    };

    ...
}
```

§ 4.13.4. `std::net::interface_preference`

Multi-valued property indicating what preference(s) should be applied to local endpoints when evaluating potential networking interfaces.

```
namespace std::net {

    ...

    class interface_preference
    {
    public:
        using value_type = std::pair<interface, transport_preference>;

        interface_preference() = default;

        interface_preference(std::initializer_list<value_type>&& values)
            : values_{ std::move(values) }
        { }

        interface_preference(interface iface, transport_preference pref)
            : values_{ std::make_pair(iface, pref) }
        { }
```

```
        using iterator = values_type::iterator;
        using const_iterator = values_type::const_iterator;

        iterator begin() noexcept;
        const_iterator begin() const noexcept;

        iterator end() noexcept;
        const_iterator end() const noexcept;

        void set(interface iface, transport_preference pref) noexcept;
        void unset(interface iface) noexcept;
    };

    ...
}
```

§ 4.13.5. `std::net::transport_props`

Nested namespace containing strongly typed properties of transports.

```
namespace std::net::transport_props {

    class interface
    {
    public:
        using value_type = interface_preference;

        // default value - interface_preference{ } a.k.a. any interface
        static value_type default_value() noexcept;
    };

    class reliability
    {
    public:
        using value_type = transport_preference;

        // default value - transport_preference::require
        static value_type default_value() noexcept;
    };

    class preserve_msg_boundaries
    {
    public:
        using value_type = transport_preference;
```

```
        // default value - transport_preference::none
        static value_type default_value() noexcept;
    };

    class per_msg_reliability
    {
    public:
        using value_type = transport_preference;

        // default value - transport_preference::none
        static value_type default_value() noexcept;
    };

    class preserve_order
    {
    public:
        using value_type = transport_preference;

        // default value - transport_preference::require
        static value_type default_value() noexcept;
    };

    class zero_rtt_msg
    {
    public:
        using value_type = transport_preference;

        // default value - transport_preference::none
        static value_type default_value() noexcept;
    };

    class multistreaming
    {
    public:
        using value_type = transport_preference;

        // default value - transport_preference::prefer
        static value_type default_value() noexcept;
    };

    class full_checksum_send
    {
    public:
        using value_type = transport_preference;

        // default value - transport_preference::require
        static value_type default_value() noexcept;
```

```
    };

    class full_checksum_recv
    {
    public:
        using value_type = transport_preference;

        // default value - transport_preference::require
        static value_type default_value() noexcept;
    };

    class congestion_control
    {
    public:
        using value_type = transport_preference;

        // default value - transport_preference::require
        static value_type default_value() noexcept;
    };

    class keep_alive
    {
    public:
        using value_type = transport_preference;

        // default value - transport_preference::none
        static value_type default_value() noexcept;
    };

    class interface
    {
    public:
        using value_type = interface_preference;

        // default value - interface_preference{ } a.k.a. any interface
        static value_type default_value() noexcept;
    };

    // TDDO class provisioning_domain; /* see [RFC7556] */

    class use_temp_local_address
    {
    public:
        using value_type = transport_preference;

        // TODO review this, it varies based on address type (e.g. ipv6 only) a
        // listeners and redezvous, prefer for rest).
        // default value - transport_preference::none
```

```
        static value_type default_value() noexcept;
    };

    class multipath
    {
    public:
        using value_type = multipath_preference;

        // TODO review this, it varies based on how connections are initiated
        // default value - transport_preference::disabled
        static value_type default_value() noexcept;
    };

    class advertises_alt_addr
    {
    public:
        using value_type = bool;

        // default value - false
        static value_type default_value() noexcept;
    };

    class direction
    {
    public:
        using value_type = direction_preference;

        // default value - direction_preference::bidirectional
        static value_type default_value() noexcept;
    };

    class soft_error_notify
    {
    public:
        using value_type = transport_preference;

        // default value - direction_preference::none
        static value_type default_value() noexcept;
    };

    class active_read_before_send
    {
    public:
        using value_type = transport_preference;

        // default value - direction_preference::none
```

```
        static value_type default_value() noexcept;
    };
}
```

§ 4.14. Properties governing transport security

§ 4.14.1. `std::net::security_props`

Nested namespace containing strongly typed security properties.

```
namespace std::net::security_props {
    // TODO Additional types defining security properties

    class allowed_protocols
    {
    public:
        using value_type = vector<string>;

        // default value - implementation defined
        static typed_erased_type default_value() noexcept;
    };

    class server_certificate
    {
    public:
        using value_type = vector<string>;
        // no default value
    };

    class client_certificate
    {
    public:
        using value_type = vector<string>;
        // no default value
    };

    class pinned_server_certificate
    {
    public:
        using value_type = vector<string>;
        // no default value
    };

    // Application Layer Protocol Negotiation [RFC7301]
```

```
class alpn_t
{
public:
    using value_type = vector<string>;
    // no default value
};

class supported_group
{
public:
    using value_type = vector<string>;
    // no default value
};

class ciphersuite
{
public:
    using value_type = vector<string>;
    // no default value
};

class signature_algorithm
{
public:
    using value_type = vector<string>;
    // no default value
};

class max_cached_sessions
{
public:
    using value_type = uint32_t;

    // default value - implementation defined
    static value_type default_value() noexcept;
}

class max_cached_sessions
{
public:
    using value_type = chrono::steady_clock::duration;

    // default value - implementation defined
    static value_type default_value() noexcept;
}

// TODO class pre_shared_key; /* value_type = key_material */
```

```
    // TODO class trust;  /* sequence sender */
    // TODO class challenge;  /* sequence sender */
}
```

§ 4.15. Obtaining Connections

§ 4.15.1. `net::initiate`

Obtain an "active open" (e.g. client) connection.

```
namespace std::net {
    _exposition only_
    namespace detail {
        struct initiate_t
        {
            template<typename Preconnection, typename Framer>
            connection-type operator()(const Preconnection&, const Framer&);

            template<typename Preconnection>
            connection-type operator()(const Preconnection&);
        };
    }
    using detail::initiate_t;
    inline constexpr initiate_t initiate{ };
}
```

§ 4.15.2. `net::listen`

Obtain a listen (e.g. server) connection

```
namespace std::net {
    _exposition only_
    namespace detail {
        struct listen_t
        {
            template<typename Preconnection, typename Framer>
            listen-type operator()(const Preconnection&, const Framer&);

            template<typename Preconnection>
```

```
            listen-type operator()(const Preconnection&);
        };
    }
    using detail::listen_t;
    inline constexpr listen_t listen{ };
}
```

§ 4.15.3. `net::rendezvous`

Obtain a peer-to-peer connection

```
namespace std::net {
    _exposition only_
    namespace detail {
        struct rendezvous_t
        {
            template<typename Preconnection, typename Framer>
            rendezvous-type operator()(const Preconnection&, const Framer&);

            template<typename Preconnection>
            rendezvous-type operator()(const Preconnection&);
        };
    }
    using detail::rendezvous_t;
    inline constexpr rendezvous_t rendezvous{ };
}
```

§ 5. Next steps

A Future paper revision to formally propose the API changes extending/modifying [p2762].

§ References

§ Informative References

[ANF]
Apple Network.framework. URL: https://developer.apple.com/documentation/network

[ASIO]

Chris Kohlhoff. *Asio C++ Library*. URL: https://think-async.com/Asio

[C-ARES]

A modern DNS resolver written in C. URL: https://c-ares.org/

[NETTS]

Working Draft, C++ Extensions for Networking. 2018-04-04. URL: https://wg21.link/n4734

[P1861]

Alex Christensen; JF Bastien; Scott Herscher. *Secure Networking in C++*. URL: https://wg21.link/p1861

[P2300]

various. *std::execution*. URL: https://wg21.link/p2300

[P2762]

Dietmar Kühl (Bloomberg). *Sender/Receiver Interface For Networking*. URL: https://wg21.link/p2762

[P3185]

Thomas Rodgers. *A proposed direction for C++ Standard Networking based on IETF TAPS*. URL: https://wg21.link/p3185

[P3325]

Eric Niebler. *A Utility for Creating Execution Environments*. URL: https://wg21.link/p3325

[TAPS_Arch]

Architecture and Requirements for Transport Services. 2023-11-09. URL: https://datatracker.ietf.org/doc/draft-ietf-taps-arch/

[TAPS_interface]

An Abstract Application Layer Interface to Transport Services. 2023-11-09. URL: https://datatracker.ietf.org/doc/draft-ietf-taps-interface/